Ψηφιακά Συστήματα VLSI

6η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

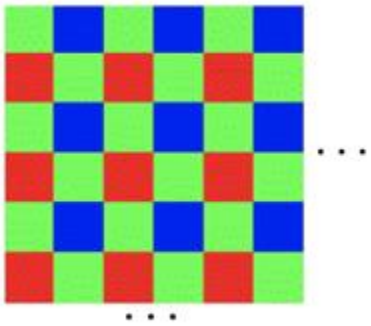
Δημήτριος Μιχαλακόπουλος (03120229)

Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

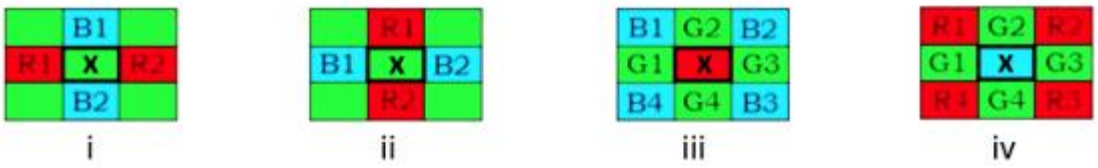
ΜΕΡΟΣ Α΄: Υλοποίηση Debayering Φίλτρου

Στα πλαίσια του πρώτου μέρους αυτής της εργαστηριακής άσκησης μας ζητείται να σχεδιάσουμε το φίλτρο Debayering, το οποίο θα μετατρέπει την Bayer εικόνα σε RGB υπολογίζοντας τους μέσους όρους των γειτονικών pixels σε 3x3 γειτονιές. Θεωρούμε ότι το μωσαϊκό Bayer ακολουθεί το πρότυπο GBRG, δηλαδή η Bayer εικόνα ακολουθεί το μοτίβο που παρουσιάζεται στο Σχήμα 2:



Σχήμα 2: Το πρότυπο GBRG της συστοιχίας χρωματικών φίλτρων Bayer [1].

Με βάση την κατανομή των χρωματικών συνιστωσών, υπάρχουν συνολικά τέσσερις διαφορετικές περιπτώσεις όπου γνωρίζουμε την μία χρωματική συνιστώσα και θέλουμε να βρούμε τις άλλες δύο. Οι περιπτώσεις αυτές παρουσιάζονται στο Σχήμα 3:

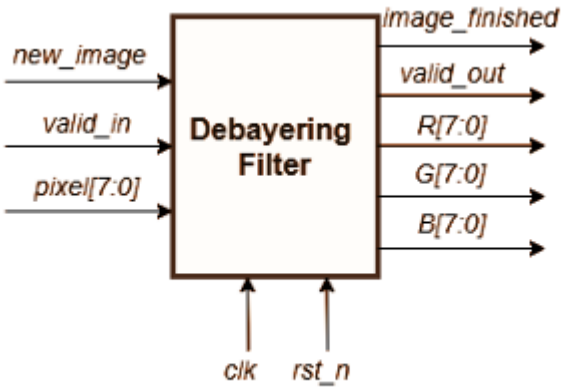


Σχημα 3: Οι πιθανές 3x3 γειτονιές της συστοιχίας χρωματικών φίλτρων Bayer.

Με βάση το Σχήμα 3, θα πρέπει να πραγματοποιηθούν οι ακόλουθοι υπολογισμοί για το κεντρικό pixel X:

- Αν αυτό είναι πράσινο (περ. i, ii), η κόκκινη (μπλε) χρωματική του συνιστώσα υπολογίζεται ως ο μέσος όρος των δύο κόκκινων (μπλε) γειτονικών pixels.
- Αν αυτό είναι κόκκινο (περ. iii), η πράσινη (μπλε) χρωματική του συνιστώσα υπολογίζεται ως ο μέσος όρος των τεσσάρων πράσινων (μπλε) γειτονικών pixels.
- Αν αυτό είναι μπλε (περ. iv), η πράσινη (κόκκινη) χρωματική του συνιστώσα υπολογίζεται ως ο μέσος όρος των τεσσάρων πράσινων (κόκκινων) γειτονικών pixels.

Ο σχεδιασμός του φίλτρου βασίζεται σε τρία modules: ένα που μετατρέπει την σειριακή σε παράλληλη είσοδο για την ανάγνωση δεδομένων, ένα που κάνει τις απαραίτητες πράξεις και ένα που χειρίζεται τα ενδιάμεσα και τα τελικά σήματα του φίλτρου. Οι είσοδοι/έξοδοι (I/O) του κορυφαίου σε ιεραρχία module της στοχευμένης αρχιτεκτονικής παρουσιάζεται στο Σχήμα 4:



Σχήμα 4: Το I/O της στοχευμένης αρχιτεκτονικής του Debayering φίλτρου.

Ακολουθεί ο κώδικας και ο σχολιασμός των βασικών modules του φίλτρου.

Serial to Parallel (S2P)

Ο κώδικας που υλοποιεί το S2P φαίνεται παρακάτω:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE IEEE.MATH_REAL.ALL;

ENTITY S2P IS
    GENERIC (N : INTEGER := 1024);
    PORT (
        clk : IN STD_LOGIC;
        rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
        pixel : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        valid_pxl : IN STD_LOGIC;
        s00 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s01 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s02 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s10 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s11 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s12 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s20 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s21 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        s22 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        valid_out : OUT STD_LOGIC
    );
END S2P;

ARCHITECTURE Behavioral OF s2p IS
    COMPONENT fifo_generator_0
        PORT (
            clk : IN STD_LOGIC;
            rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
            din : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
            wr_en : IN STD_LOGIC;
            rd_en : IN STD_LOGIC;
            dout : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            full : OUT STD_LOGIC;
            empty : OUT STD_LOGIC;
            valid : OUT STD_LOGIC;
            data_count : OUT STD_LOGIC_VECTOR(9 DOWNTO 0)
        );
    END COMPONENT;

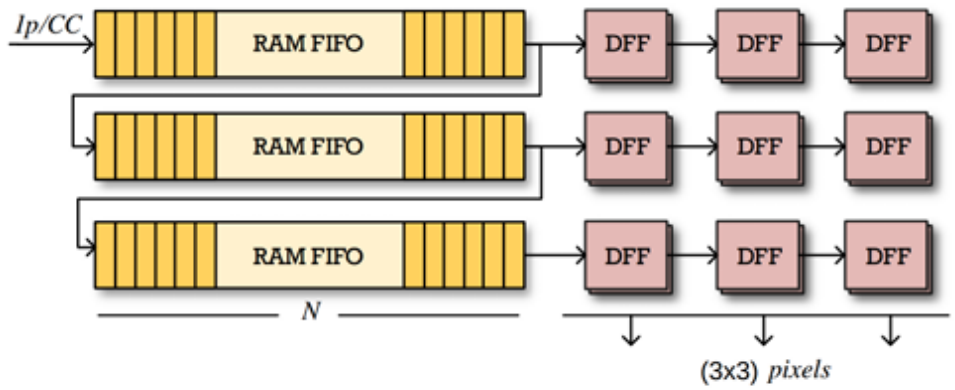
    TYPE arr IS ARRAY (2 DOWNTO 0) OF STD_LOGIC_VECTOR (7 DOWNTO 0);
    SIGNAL L0, L1, L2 : arr := (OTHERS => (OTHERS => '0'));
    SIGNAL counter : STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N * N)))) DOWNTO 0) := (OTHERS => '0');
    SIGNAL out0, out1, out2 : STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');
    SIGNAL rd_en0, rd_en1, rd_en2 : STD_LOGIC := '0';
    SIGNAL full0, full1, full2, empty0, empty1, empty2, valid0, valid1, valid2 : STD_LOGIC;
    SIGNAL data_count0, data_count1, data_count2 : STD_LOGIC_VECTOR(9 DOWNTO 0);
```

```
50
51  BEGIN
52      RAM_FIFO_0 : fifo_generator_0
53      PORT MAP(
54          clk => clk,
55          rst_n => rst_n,
56          din => pixel,
57          wr_en => '1',
58          rd_en => rd_en0,
59          dout => out0,
60          full => full0,
61          empty => empty0,
62          valid => valid0,
63          data_count => data_count0
64      );
65      RAM_FIFO_1 : fifo_generator_0
66      PORT MAP(
67          clk => clk,
68          rst_n => rst_n,
69          din => out0,
70          wr_en => '1',
71          rd_en => rd_en1,
72          dout => out1,
73          full => full1,
74          empty => empty1,
75          valid => valid1,
76          data_count => data_count1
77      );
78      RAM_FIFO_2 : fifo_generator_0
79      PORT MAP(
80          clk => clk,
81          rst_n => rst_n,
82          din => out1,
83          wr_en => '1',
84          rd_en => rd_en2,
85          dout => out2,
86          full => full2,
87          empty => empty2,
88          valid => valid2,
89          data_count => data_count2
90      );
91
92      PROCESS (clk, rst_n)
93      BEGIN
94          IF rst_n(0) = '1' THEN
95              rd_en0 <= '0';
96              rd_en1 <= '0';
97              rd_en2 <= '0';
98              valid_out <= '0';
```

```
            counter <= (OTHERS => '0');
        ELSIF rising_edge(clk) THEN
            IF valid_pxl = '1' THEN
                counter <= counter + 1;
                IF to_integer(unsigned(counter)) = 2 * N + 3 THEN
                    valid_out <= '1';
                END IF;
                IF to_integer(unsigned(data_count0)) = N - 2 THEN
                    rd_en0 <= '1';
                    rd_en1 <= '1';
                    rd_en2 <= '1';
                END IF;
                L0 <= L0(1 DOWNTO 0) & out0;
                L1 <= L1(1 DOWNTO 0) & out1;
                L2 <= L2(1 DOWNTO 0) & out2;
                s00 <= L0(0);s01 <= L0(1);s02 <= L0(2);
                s10 <= L1(0);s11 <= L1(1);s12 <= L1(2);
                s20 <= L2(0);s21 <= L2(1);s22 <= L2(2);
            END IF;
        END IF;
    END PROCESS;

END Behavioral;
```

Η υλοποίηση του S2P βασίζεται σε τρεις RAM FIFO και σε εννέα D flip-flops όπως άλλωστε φαίνεται και στην προτεινόμενη αρχιτεκτονική στο Σχήμα 5:



Σχήμα 5: Η προτεινόμενη αρχιτεκτονική του μετατροπέα σειριακού-σε-παράλληλο.

Για τις RAM χρησιμοποιήσαμε το έτοιμο IP του Vivado επιλέγοντας στο generator Read/Write Width: 8 και Read/Write Depth: 1024. Επίσης, για να λειτουργεί το φίλτρο παραμετρικά, με την έξοδο datacount μετράμε το πλήθος δεδομένων της FIFO. Έπειτα, για να πετύχουμε τον κατάλληλο συγχρονισμό με τους καταχωρητές αρχίζουμε να διαβάζουμε από τις RAM όταν η πρώτη από αυτές περιέχει N-2 δεδομένα. Στην συνέχεια, με την βοήθεια ενός ακόμα counter εμφανίζουμε το σήμα valid_out όταν η τιμή του πρώτου pixel φτάσει στο κεντρικό DFF, άρα για επιπλέον N+4 κύκλους ρολογιού, δηλαδή συνολικά για: (N-2) + (N+4) + 1= 2N+3 κύκλους ρολογιού (ένας επιπλέον κύκλος για συγχρονισμό με το compute unit).

Compute Unit

Ο κώδικας που υλοποιεί το Compute Unit φαίνεται παρακάτω:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE IEEE.MATH_REAL.ALL;

ENTITY COMPUTE_UNIT IS
    GENERIC (N : INTEGER := 1024);
    PORT (
        clk : IN STD_LOGIC;
        rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
        s00 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s01 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s02 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s10 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s11 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s12 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s20 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s21 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        s22 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        lline : IN STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
        column : IN STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
        R : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        G : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        B : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
    );
END COMPUTE_UNIT;

ARCHITECTURE Behavioral OF COMPUTE_UNIT IS

    SIGNAL downright, downleft, upright, upleft, right, left, center, up, down : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL validout : STD_LOGIC;
    SIGNAL w0, w1 : STD_LOGIC_VECTOR(9 DOWNTO 0);
    SIGNAL w2, w3 : STD_LOGIC_VECTOR(8 DOWNTO 0);
    SIGNAL up_down : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL left_right : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL corners : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL cross : STD_LOGIC_VECTOR(7 DOWNTO 0);

BEGIN

    w0 <= ("00" & left) + ("00" & right) + ("00" & up) + ("00" & down);
    w1 <= ("00" & upleft) + ("00" & upright) + ("00" & downleft) + ("00" & downright);
    w2 <= ('0' & up) + ('0' & down);
    w3 <= ('0' & left) + ('0' & right);
    cross <= w0(9 DOWNTO 2);
    corners <= w1(9 DOWNTO 2);
    up_down <= w2(8 DOWNTO 1);
    left_right <= w3(8 DOWNTO 1);
```

```
    upleft <= s22 WHEN to_integer(unsigned(lline)) /= 0 AND to_integer(unsigned(column)) /= 0 ELSE (OTHERS => '0');
    up <= s21 WHEN to_integer(unsigned(lline)) /= 0 ELSE (OTHERS => '0');
    upright <= s20 WHEN to_integer(unsigned(lline)) /= 0 AND to_integer(unsigned(column)) /= N - 1 ELSE (OTHERS => '0');
    left <= s12 WHEN to_integer(unsigned(column)) /= 0 ELSE (OTHERS => '0');
    center <= s11;
    right <= s10 WHEN to_integer(unsigned(column)) /= N - 1 ELSE (OTHERS => '0');
    downleft <= s02 WHEN to_integer(unsigned(lline)) /= N - 1 AND to_integer(unsigned(column)) /= 0 ELSE (OTHERS => '0');
    down <= s01 WHEN to_integer(unsigned(lline)) /= N - 1 ELSE (OTHERS => '0');
    downright <= s00 WHEN to_integer(unsigned(lline)) /= N - 1 AND to_integer(unsigned(column)) /= N - 1 ELSE (OTHERS => '0');

    PROCESS (clk, rst_n)
    BEGIN
        IF rising_edge(clk) THEN
            IF lline(0) = '0' THEN
                IF column(0) = '0' THEN --green
                    R <= up_down;
                    G <= center;
                    B <= left_right;
                ELSE --blue
                    R <= corners;
                    G <= cross;
                    B <= center;
                END IF;
            ELSE
                IF column(0) = '0' THEN --red
                    R <= center;
                    G <= cross;
                    B <= corners;
                ELSE --green
                    R <= left_right;
                    G <= center;
                    B <= up_down;
                END IF;
            END IF;
        END IF;
    END PROCESS;
END Behavioral;
```

Σε αυτό το module γίνονται όλες οι απαραίτητες πράξεις και με τα σήματα που στέλνει το control unit εμφανίζονται στην έξοδο οι τιμές RGB των pixel. Αρχικά, σημαντική παρατήρηση είναι ότι, λόγω του ότι οι τιμές των pixel διαβάζονται ως data stream, τα αρχικά inputs καταλήγουν να βρίσκονται στις τελευταίες γραμμές και στήλες της 3x3 γειτονίας του S2P module, άρα στο compute unit θα πρέπει αυτός ο πίνακας που παίρνουμε για είσοδο να αναδιαμορφώνεται σύμφωνα με την σχέση: S2P(i, j) = Compute(2-i, 2-j), όπου $i,j \in [0,2]$. Συνεπώς, λοιπόν, οι τιμές που προκύπτουν από το S2P module τοποθετούνται στην σωστή θέση έτσι ώστε να σχηματιστούν οι κατάλληλες 3x3 γειτονίες και έπειτα γίνεται ο έλεγχος για τις ακραίες περιπτώσεις όπου κάποιες θέσεις της γειτονιάς πρέπει να μηδενιστούν. Στην συνέχεια, γίνεται ο υπολογισμός των τεσσάρων μέσων όρων: πάνω-κάτω, δεξιά-αριστερά, πάνω-κάτω-δεξιά-αριστερά, πάνω-κάτω-δεξιά-αριστερά άκρες. Τέλος, ελέγχουμε σε ποιον συνδυασμό γραμμής και στήλης της εικόνας είμαστε προκειμένου να αντιστοιχίσουμε κάθε γειτονία σε μία από τις 4 περιπτώσεις που φαίνονται στο Σχήμα 3 και να εμφανίσουμε τις κατάλληλες τιμές RGB για κάθε μία από αυτές.

Control Unit

Ο κώδικας που υλοποιεί το Control Unit φαίνεται παρακάτω:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE IEEE.MATH_REAL.ALL;

ENTITY CONTROL IS
    GENERIC (
        N : INTEGER := 1024
    );
    PORT (
        clk : IN STD_LOGIC;
        rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
        valid_in : IN STD_LOGIC;
        new_image : IN STD_LOGIC;
        start : IN STD_LOGIC;
        lline : OUT STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
        column : OUT STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
        valid_out : OUT STD_LOGIC;
        image_finished : OUT STD_LOGIC
    );
END CONTROL;

ARCHITECTURE Behavioral OF CONTROL IS

    SIGNAL ColReg, LineReg : STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0) := (OTHERS => '0');
BEGIN

    column <= ColReg;
    lline <= LineReg;

    PROCESS (clk, rst_n)
    BEGIN
        IF rst_n(0) = '1' THEN
            LineReg <= (OTHERS => '0');
            ColReg <= (OTHERS => '0');
            valid_out <= '0';
        ELSIF rising_edge(clk) THEN
            valid_out <= '0';
            IF (start = '1') THEN
                valid_out <= '1';
                ColReg <= ColReg + 1;
                IF to_integer(unsigned(ColReg)) = N - 1 THEN
                    LineReg <= LineReg + 1;
                END IF;
            END IF;
            IF (to_integer(unsigned(LineReg)) = N - 1 AND to_integer(unsigned(ColReg)) = N - 1) THEN
                image_finished <= '1';
            ELSE
                image_finished <= '0';
            END IF;
        END IF;
    END PROCESS;
END Behavioral;
```

Σε αυτό το module γίνεται ο έλεγχος της ορθής λειτουργίας του κυκλώματος και στέλνουμε τα κατάλληλα σήματα στο compute unit ώστε να γίνει ο υπολογισμός

και να εμφανιστούν στην έξοδο του φίλτρου τα αποτελέσματα του Debayering. Συγκεκριμένα, όταν λάβουμε την πρώτη έγκυρη έξοδο από το S2P module, τότε αρχίζουμε να αυξάνουμε την γραμμή και την στήλη της εικόνας ανάλογα με την θέση του pixel που βρισκόμαστε ώστε να πραγματοποιηθούν οι σωστές πράξεις. Τέλος, όταν βρεθούμε στο τελευταίο pixel της εικόνας πρέπει να εμφανίσουμε το αντίστοιχο σήμα ότι τελείωσε το Debayering της εικόνας (image_finished).

Debayering Filter

Ο κώδικας που υλοποιεί το Debayering Filter φαίνεται παρακάτω:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE IEEE.MATH_REAL.ALL;

ENTITY DEBAYERING IS
    GENERIC (N : INTEGER := 1024);
    PORT (
        clk : IN STD_LOGIC;
        rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
        valid_in : IN STD_LOGIC;
        pixel : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        new_image : IN STD_LOGIC;
        R : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        G : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        B : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
        valid_out : OUT STD_LOGIC;
        image_finished : OUT STD_LOGIC
    );
END DEBAYERING;

ARCHITECTURE Behavioral OF DEBAYERING IS

    COMPONENT s2p
        GENERIC (N : INTEGER);
        PORT (
            clk : IN STD_LOGIC;
            rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
            pixel : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
            valid_pxl : IN STD_LOGIC;
            s00 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s01 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s02 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s10 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s11 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s12 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s20 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s21 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            s22 : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            valid_out : OUT STD_LOGIC
        );
    END COMPONENT;
```

```
44
45      COMPONENT COMPUTE_UNIT
46          GENERIC (N : INTEGER);
47          PORT (
48              clk : IN STD_LOGIC;
49              rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
50              s00 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
51              s01 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
52              s02 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
53              s10 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
54              s11 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
55              s12 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
56              s20 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
57              s21 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
58              s22 : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
59              lline : IN STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
60              column : IN STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
61              R : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
62              G : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
63              B : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
64          );
65      END COMPONENT;
66
67      COMPONENT CONTROL
68          GENERIC (N : INTEGER);
69          PORT (
70              clk : IN STD_LOGIC;
71              rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
72              valid_in : IN STD_LOGIC;
73              new_image : IN STD_LOGIC;
74              start : IN STD_LOGIC;
75              lline : OUT STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
76              column : OUT STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0);
77              valid_out : OUT STD_LOGIC;
78              image_finished : OUT STD_LOGIC
79          );
80      END COMPONENT;
81      SIGNAL clk_gate : STD_LOGIC;
82      SIGNAL ss00, ss01, ss02, ss10, ss11, ss12, ss20, ss21, ss22 : STD_LOGIC_VECTOR(7 DOWNTO 0);
83      SIGNAL lin, col : STD_LOGIC_VECTOR(INTEGER(ceil(log2(real(N)))) - 1 DOWNTO 0) := (OTHERS => '0');
84      SIGNAL start : STD_LOGIC;
85      SIGNAL valid_out1 : STD_LOGIC;
86
87  BEGIN
88
89      CTRL : CONTROL
90      GENERIC MAP(N => N)
91      PORT MAP(
92          clk => clk_gate,
93          rst_n => rst_n,
94          valid_in => valid_in,
95          new_image => new_image,
96          start => start,
97          lline => lin,
98          column => col,
99          valid_out => valid_out1,
100         image_finished => image_finished
101     );
```

```
102
103     CU : COMPUTE_UNIT
104     GENERIC MAP(N => N)
105     PORT MAP(
106         clk => clk_gate,
107         rst_n => rst_n,
108         s00 => ss00,
109         s01 => ss01,
110         s02 => ss02,
111         s10 => ss10,
112         s11 => ss11,
113         s12 => ss12,
114         s20 => ss20,
115         s21 => ss21,
116         s22 => ss22,
117         lline => lin,
118         column => col,
119         R => R,
120         G => G,
121         B => B
122     );
123
124     S_2_P : S2P
125     GENERIC MAP(N => N)
126     PORT MAP(
127         clk => clk_gate,
128         rst_n => rst_n,
129         pixel => pixel,
130         valid_pxl => '1',
131         s00 => ss00,
132         s01 => ss01,
133         s02 => ss02,
134         s10 => ss10,
135         s11 => ss11,
136         s12 => ss12,
137         s20 => ss20,
138         s21 => ss21,
139         s22 => ss22,
140         valid_out => start
141     );
142
143     valid_out <= valid_out1 AND valid_in;
144     clk_gate <= (clk AND valid_in);
145
146 END Behavioral;
```

Αυτό είναι το top module του Debayering φίλτρου στο οποίο πραγματοποιείται η σύνδεση όλων των επιμέρους κυκλωμάτων. Επίσης, εδώ διαμορφώνουμε σωστά το σήμα έγκυρης εξόδου (valid_out) ώστε να προκύπτει όταν έχουμε ταυτόχρονα έγκυρη είσοδο στο φίλτρο και έγκυρη έξοδο στο control unit. Οι είσοδοι και οι έξοδοι σχεδιάστηκαν βασισμένοι στην αρχιτεκτονική που παρουσιάζεται στο Σχήμα 4 που υπάρχει παραπάνω.

Testbench

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;
USE STD.TEXTIO.ALL;

ENTITY DEBAYERING_tb IS
END DEBAYERING_tb;

ARCHITECTURE tb_arch OF DEBAYERING_tb IS
    CONSTANT N : INTEGER := 32;

    COMPONENT DEBAYERING
        GENERIC (N : INTEGER);
        PORT (
            clk : IN STD_LOGIC;
            rst_n : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
            new_image : IN STD_LOGIC;
            valid_in : IN STD_LOGIC;
            pixel : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
            R : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            G : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            B : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
            valid_out : OUT STD_LOGIC;
            image_finished : OUT STD_LOGIC
        );
    END COMPONENT;

    SIGNAL clk : STD_LOGIC := '0';
    SIGNAL rst_n : STD_LOGIC_VECTOR(0 DOWNTO 0) := (OTHERS => '0');
    SIGNAL new_image : STD_LOGIC := '0';
    SIGNAL valid_in : STD_LOGIC := '0';
    SIGNAL pixel : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL enable : STD_LOGIC := '0';
    SIGNAL R, G, B : STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL valid_out : STD_LOGIC;
    SIGNAL image_finished : STD_LOGIC;
    CONSTANT CLK_PERIOD : TIME := 1ns;

BEGIN
    UUT : DEBAYERING
    GENERIC MAP(N => N)
    PORT MAP(
        clk => clk,
        rst_n => rst_n,
        new_image => new_image,
        valid_in => valid_in,
        pixel => pixel,
        R => R,
        G => G,
```

```
        B => B,
        valid_out => valid_out,
        image_finished => image_finished
    );

    clk_proc : PROCESS
    BEGIN
        clk <= '0';
        WAIT FOR CLK_PERIOD / 2;
        clk <= '1';
        WAIT FOR CLK_PERIOD / 2;
    END PROCESS;

    init : PROCESS
    BEGIN
        WAIT FOR clk_period/2;
        rst_n <= "0";
        enable <= '1';
        valid_in <= '1';
        WAIT FOR 1048576 * clk_period;
        valid_in <= '0';
        WAIT FOR 10 * clk_period;
        WAIT FOR 2 * clk_period;
        WAIT;
    END PROCESS;

    read_proc : PROCESS (clk, rst_n)
        FILE input_file : text OPEN read_mode IS "C:\Users\Dimitris\Desktop\read.txt";
        VARIABLE row_read : line;
        VARIABLE pixel_read : INTEGER;
        VARIABLE pixel_row_counter : INTEGER := 0;
        FILE output_file : text OPEN write_mode IS "C:\Users\Dimitris\Desktop\write.txt";
        VARIABLE row_write : line;

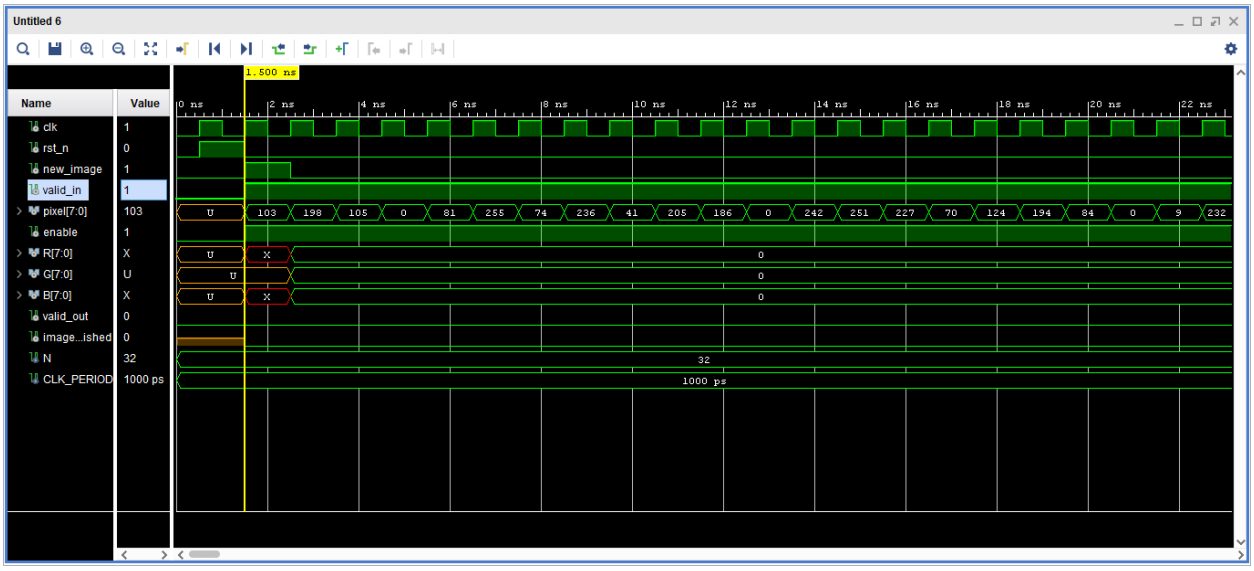
    BEGIN
        IF (rst_n = "1") THEN
            pixel_row_counter := 0;
            pixel_read := - 1;
        ELSIF (rising_edge(clk)) THEN
            IF (enable = '1') THEN
                IF (NOT endfile(input_file)) THEN
                    IF (pixel_row_counter = 0) THEN
                        new_image <= '1';
                    ELSE
                        IF (pixel_row_counter = N * N - 1) THEN
                            pixel_row_counter := - 1;
                        END IF;
                        new_image <= '0';
                    END IF;
```

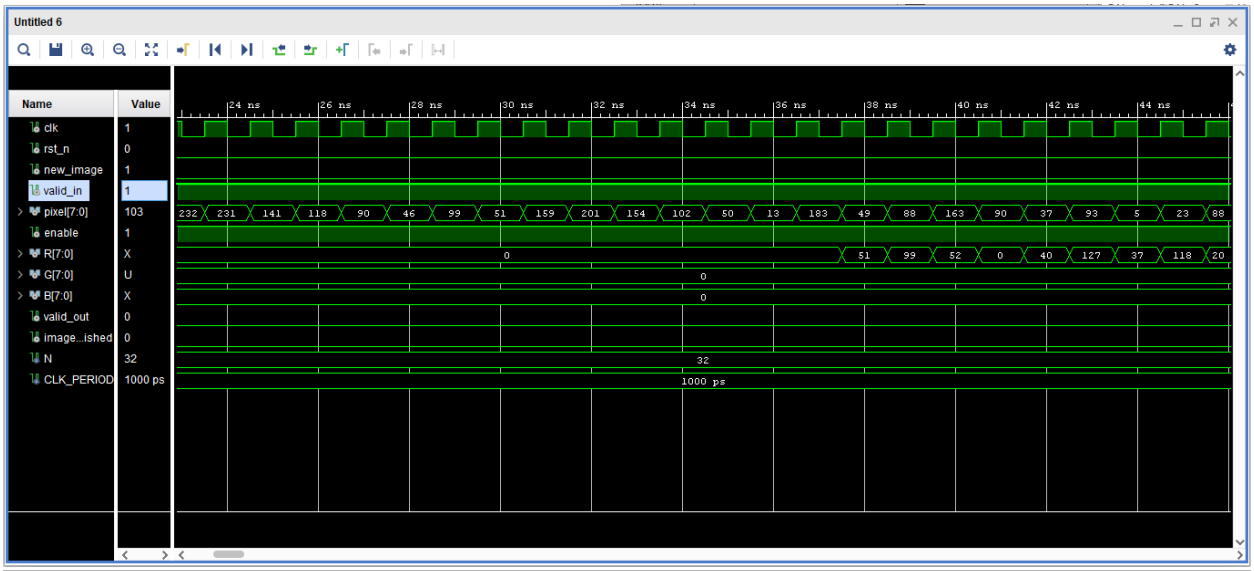
```
                    pixel_row_counter := pixel_row_counter + 1;
                    readline(input_file, row_read);
                END IF;
                read(row_read, pixel_read);
                pixel <= STD_LOGIC_VECTOR(to_unsigned(pixel_read, 8));
            END IF;
            IF (valid_out = '1') THEN
                write(row_write, to_integer(unsigned(R)), left, 4);
                write(row_write, to_integer(unsigned(G)), left, 4);
                write(row_write, to_integer(unsigned(B)), left, 4);
                writeline(output_file, row_write);
            END IF;
            IF (image_finished = '1' AND valid_out = '1') THEN
                write(row_write, STRING'("<Image Finished>"));
                writeline(output_file, row_write);
            END IF;
        END IF;
    END PROCESS;

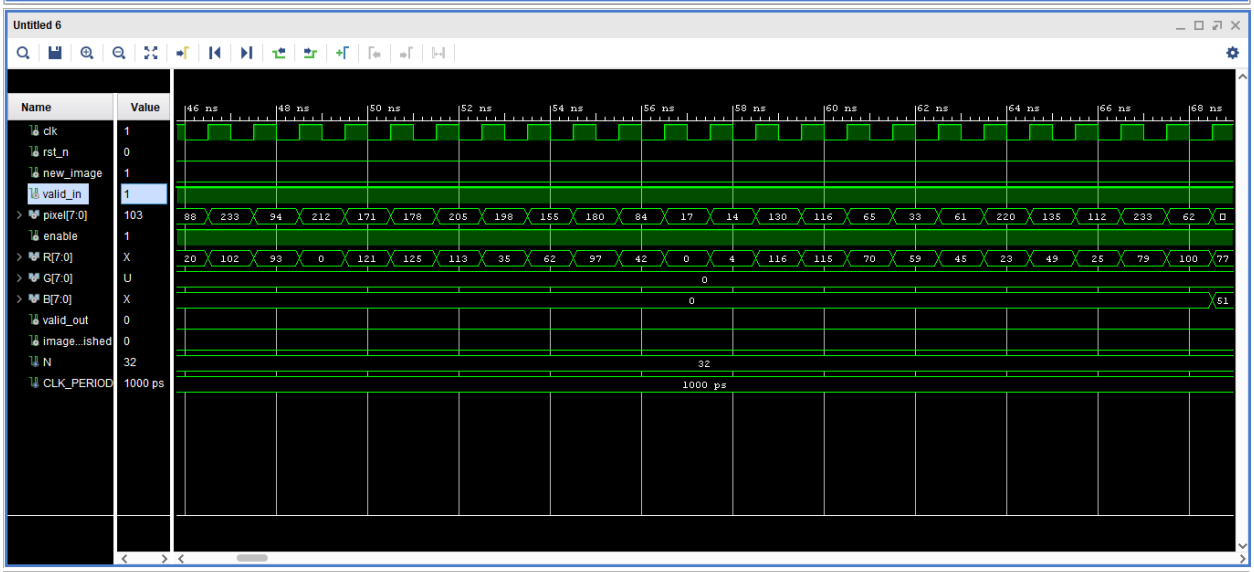
END tb_arch;
```

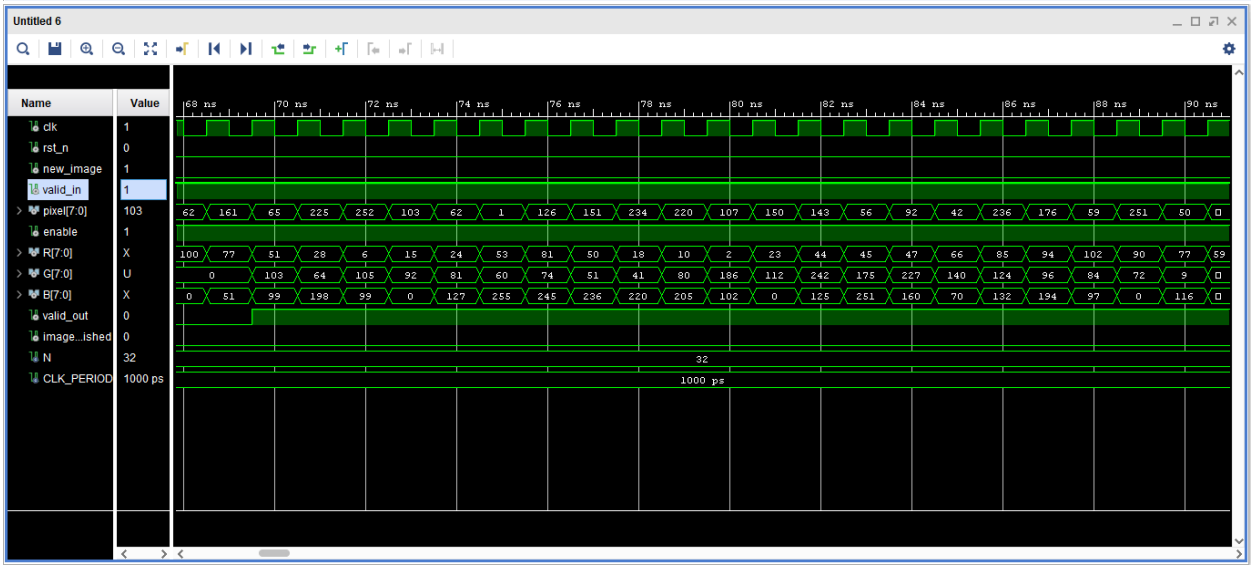
Το παραπάνω testbench εφαρμόζει στο Debayering φίλτρο για μέγεθος εικόνας N=32 τα δεδομένα ''TEST INPUT DATA'' από το Helios τα οποία έχουμε περάσει στο αρχείο read.txt και βγάζει τις τιμές RGB για κάθε pixel στο αρχείο write.txt. Παράλληλα τα ίδια αποτελέσματα εμφανίζονται στην προσομοίωση του φίλτρου.

Η προσομοίωση του Debayering φίλτρου φαίνεται παρακάτω:




Untitled 6
Name
Value
clk
rst_n
new_image
valid_in
pixel[7:0]
enable
R[7:0]
G[7:0]
B[7:0]
valid_out
image...ished
N
CLK_PERIOD
1
0
1
1
103
1
X
U
X
0
0
32
1000 ps
24 ns
26 ns
28 ns
30 ns
32 ns
34 ns
36 ns
38 ns
40 ns
42 ns
44 ns
232 231 141 118 90 46 99 51 159 201 154 102 50 13 183 49 88 163 90 37 93 5 23 88
0 51 99 52 0 40 127 37 118 20


Untitled 6
Name
Value
clk
rst_n
new_image
valid_in
pixel[7:0]
enable
R[7:0]
G[7:0]
B[7:0]
valid_out
image...ished
N
CLK_PERIOD
1
0
1
1
103
1
X
U
X
0
0
32
1000 ps
46 ns
48 ns
50 ns
52 ns
54 ns
56 ns
58 ns
60 ns
62 ns
64 ns
66 ns
68 ns
88 233 94 212 171 178 205 198 155 180 84 17 14 130 116 65 33 61 220 135 112 233 62
20 102 93 0 121 125 113 35 62 97 42 0 4 116 115 70 59 45 23 49 25 79 100 77
51


Untitled 6
Name
Value
clk
rst_n
new_image
valid_in
pixel[7:0]
enable
R[7:0]
G[7:0]
B[7:0]
valid_out
image...ished
N
CLK_PERIOD
1
0
1
1
103
1
X
U
X
0
0
32
1000 ps
68 ns
70 ns
72 ns
74 ns
76 ns
78 ns
80 ns
82 ns
84 ns
86 ns
88 ns
90 ns
62 161 65 225 252 103 62 1 126 151 234 220 107 150 143 56 92 42 236 176 59 251 50
100 77 51 28 6 15 24 53 81 50 18 10 2 23 44 45 47 66 85 94 102 90 77 59
0 103 64 105 92 81 60 74 51 41 80 186 112 242 175 227 140 124 96 84 72 9
0 51 99 198 99 0 127 255 245 236 220 205 102 0 125 251 160 70 132 194 97 0 116

Debayering Software in C

Το πρόγραμμα σε C που κάνει την αντίστοιχη διαδικασία φαίνεται παρακάτω:

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int N;
    FILE *inputFile = fopen("input.txt", "r");
    FILE *outputFile = fopen("output.txt", "w");

    printf("Give me N: ");
    scanf("%d", &N);
    int input[N][N];
    int output[N][N][3];

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            fscanf(inputFile, "%d", &input[i][j]);
        }
    }

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            for (int i = 0; i < N; i++)
            {
                for (int j = 0; j < N; j++)
                {
                    int right = (j < N - 1) ? j + 1 : 0;
                    int left = (j > 0) ? j - 1 : 0;
                    int up = (i > 0) ? i - 1 : 0;
                    int down = (i < N - 1) ? i + 1 : 0;

                    int r = (j < N - 1) ? input[i][right] : 0;
                    int l = (j > 0) ? input[i][left] : 0;
                    int u = (i > 0) ? input[up][j] : 0;
                    int d = (i < N - 1) ? input[down][j] : 0;
                    int d1 = (i > 0 && j > 0) ? input[up][left] : 0;
                    int d2 = (i > 0 && j < N - 1) ? input[up][right] : 0;
                    int d3 = (i < N - 1 && j > 0) ? input[down][left] : 0;
                    int d4 = (i < N - 1 && j < N - 1) ? input[down][right] : 0;

                    if (i % 2)
                    {
                        if (j % 2)
                        {
                            output[i][j][0] = (r + l) / 2;
```

```
                    output[i][j][1] = input[i][j];
                    output[i][j][2] = (u + d) / 2;
                }
                else
                {
                    output[i][j][0] = input[i][j];
                    output[i][j][1] = (u + d + r + l) / 4;
                    output[i][j][2] = (d1 + d2 + d3 + d4) / 4;
                }
            }
            else
            {
                if (j % 2)
                {
                    output[i][j][0] = (d1 + d2 + d3 + d4) / 4;
                    output[i][j][1] = (u + d + r + l) / 4;
                    output[i][j][2] = input[i][j];
                }
                else
                {
                    output[i][j][0] = (u + d) / 2;
                    output[i][j][1] = input[i][j];
                    output[i][j][2] = (r + l) / 2;
                }
            }
        }
    }
    printf("\n");
}

for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
    {
        fprintf(outputFile, "%d %d %d\n", output[i][j][0], output[i][j][1], output[i][j][2]);
    }
}

return 0;
}
```

Στο παραπάνω πρόγραμμα θέτουμε ως είσοδο τα δεδομένα "TEST INPUT DATA" στο αρχείο input.txt και έπειτα οι τιμές RGB των pixel γράφονται στο output.txt. Συγκρίνοντας τα αποτελέσματα του software και του hardware βλέπουμε ότι έχουμε πλήρη ταύτιση άρα η υλοποίηση του Debayering φίλτρου είναι ορθή.

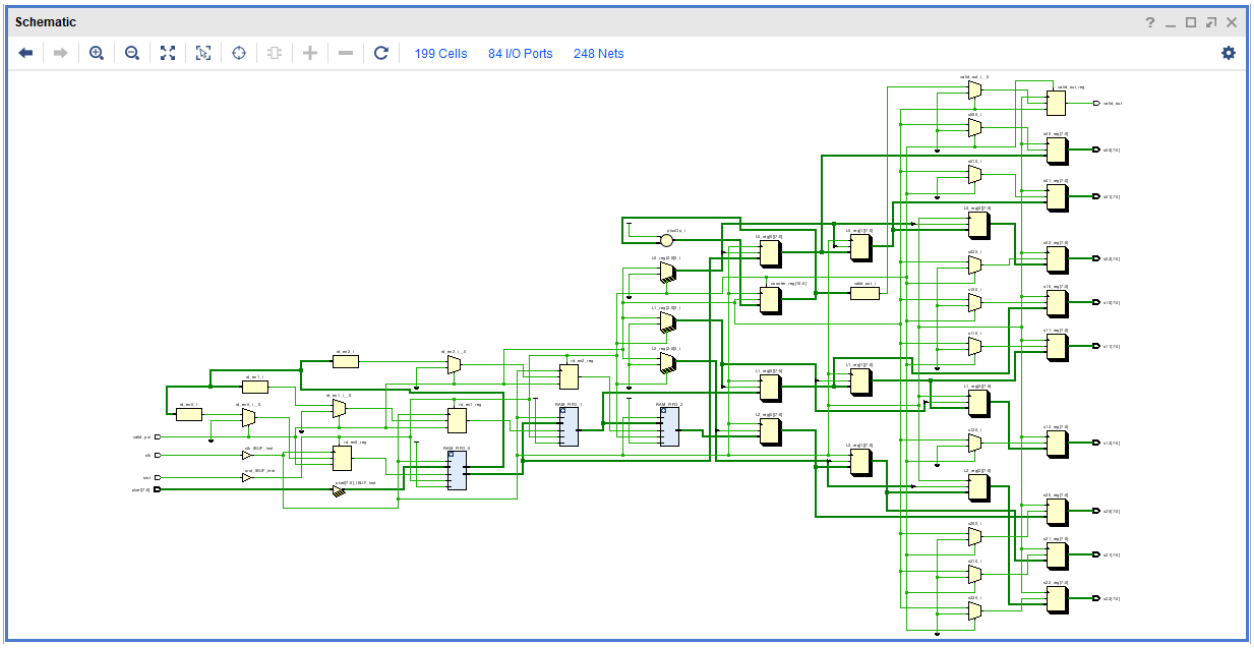
Latency and Throughput

Θέλουμε 2N+3 κύκλους ρολογιού για το πρώτο pixel στο S2P και ακόμη 2 κύκλους ρολογιού για να φτάσει στο compute unit άρα το κύκλωμα έχει: $\mathbf{T_{latency}= 2N+5\ cc}$.

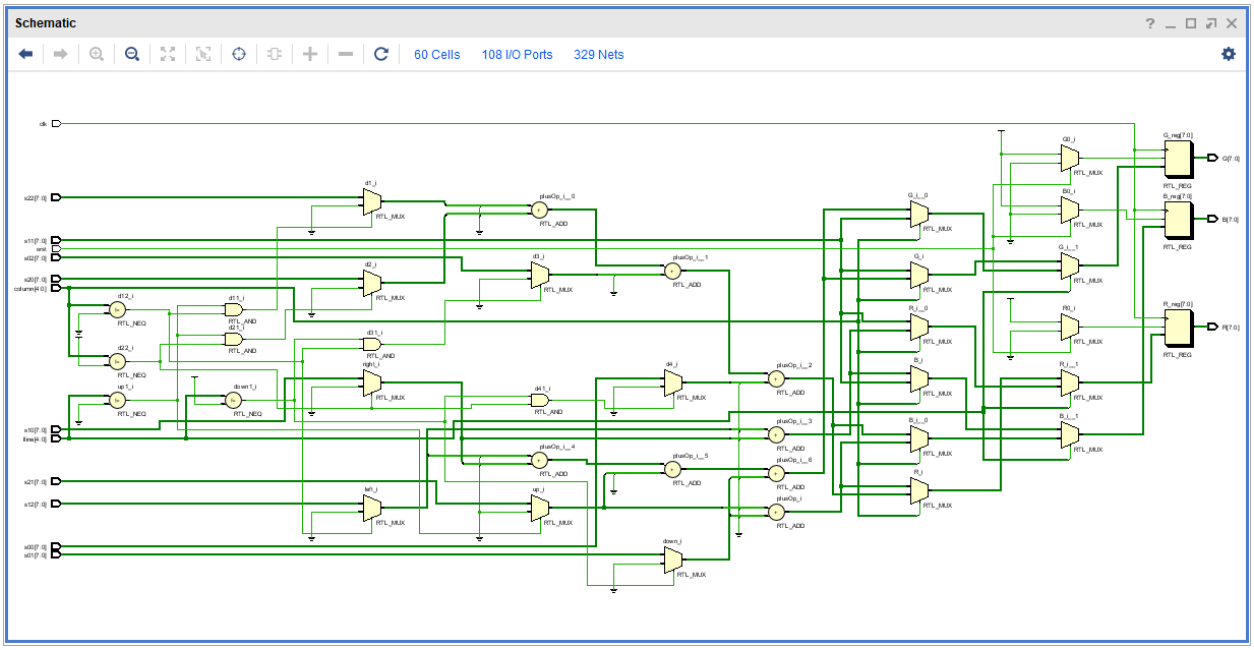
Επίσης, έχουμε μία έγκυρη έξοδο σε κάθε κύκλο ρολογιού άρα: **Throughput = 1**.

Block Diagrams

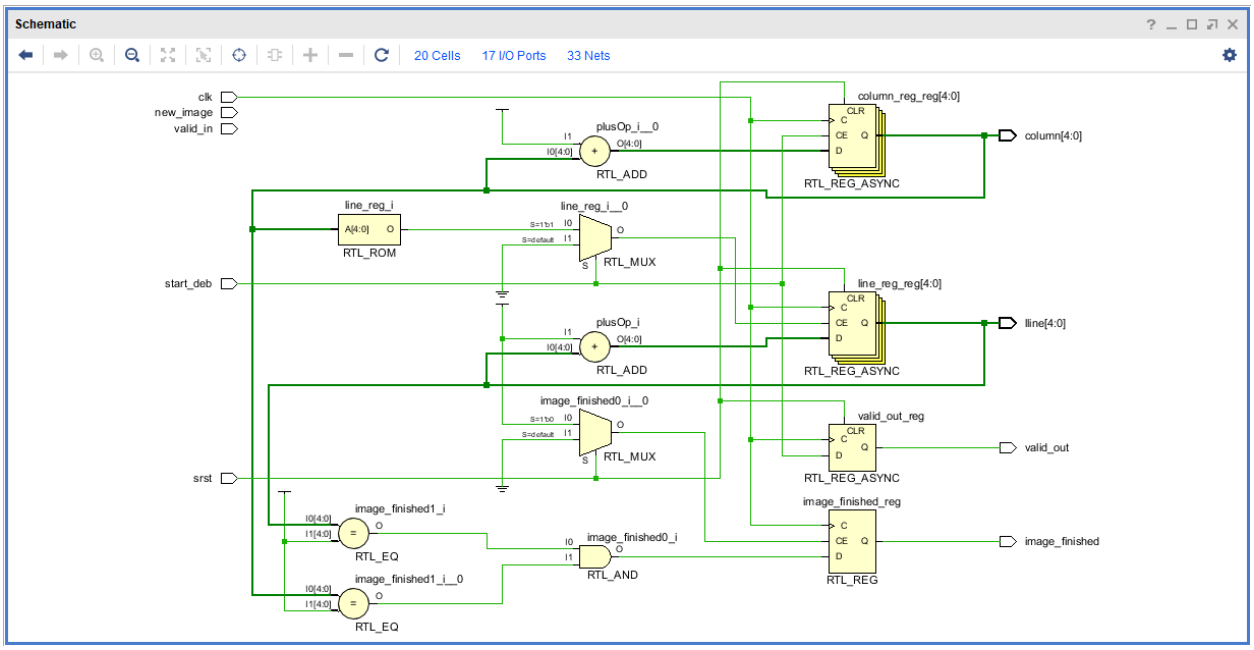
Το RTL σχηματικό του S2P φαίνεται παρακάτω:



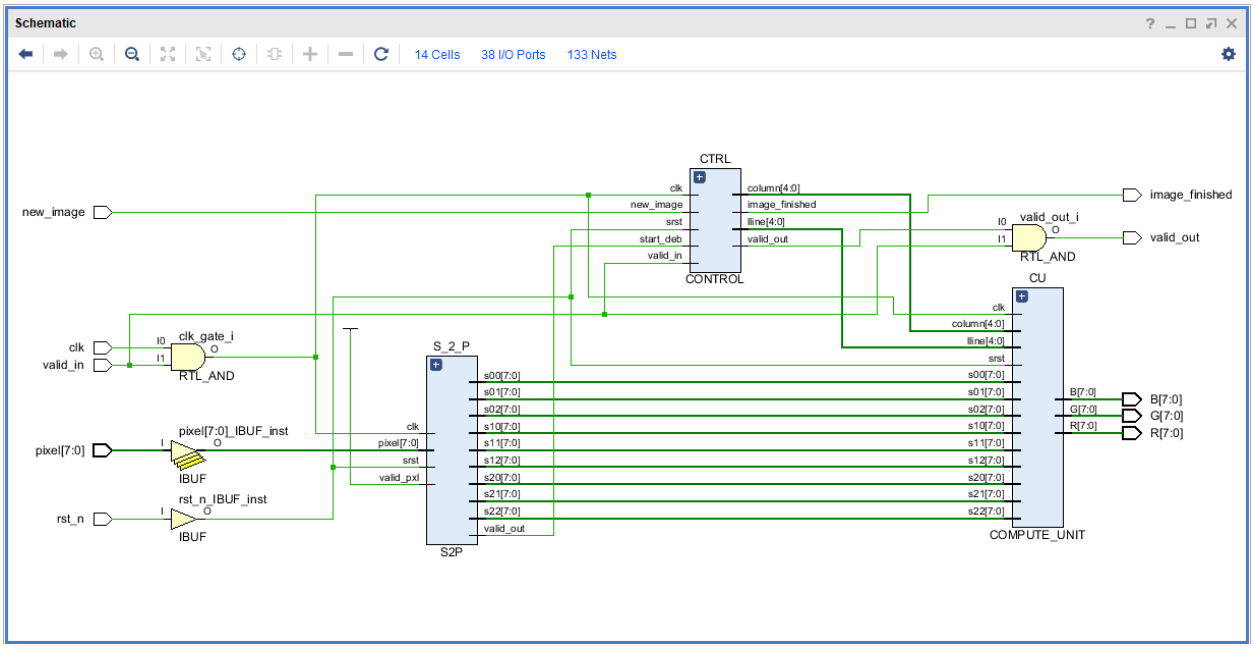
Το RTL σχηματικό του Compute Unit φαίνεται παρακάτω:



Το RTL σχηματικό του Control Unit φαίνεται παρακάτω:



Το RTL σχηματικό του Debayering Filter φαίνεται παρακάτω:



FPGA Resource Utilization

Οι πόροι που χρησιμοποιήθηκαν για N=64 φαίνονται παρακάτω:

Resource	Utilization	Available	Utilization %
LUT	216	17600	1.23
FF	242	35200	0.69
BRAM	1.50	60	2.50
IO	37	100	37.00
BUFG	1	32	3.13

Οι πόροι που χρησιμοποιήθηκαν για N=128 φαίνονται παρακάτω:

Resource	Utilization	Available	Utilization %
LUT	239	17600	1.36
FF	259	35200	0.74
BRAM	1.50	60	2.50
IO	37	100	37.00
BUFG	1	32	3.13

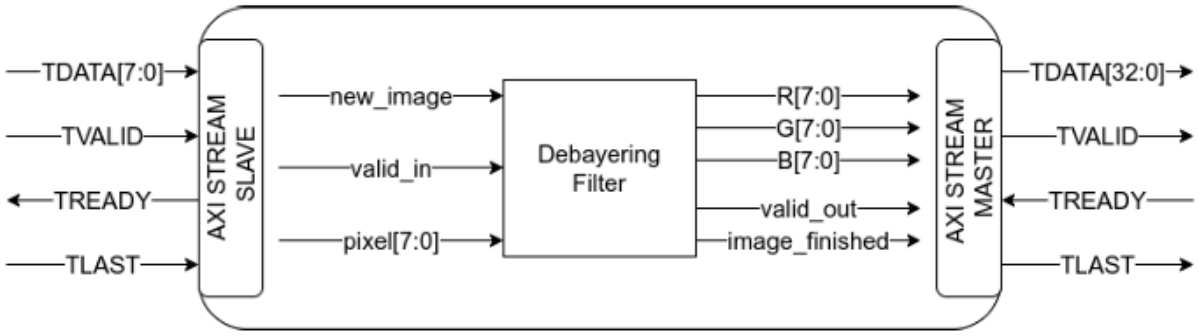
Οι διαφορές που παρατηρούνται στους πόρους που χρησιμοποιούνται είναι πάρα πολύ μικρές, ενώ την μεγαλύτερη χρήση έχουν οι Θύρες Εισόδου/Εξόδου.

Όλοι οι παραπάνω VHDL κώδικες των components και του testbench, καθώς και ο κώδικας του προγράμματος επαλήθευσης σε C βρίσκονται μέσα στο zip αρχείο.

ΜΕΡΟΣ Β΄: Προγραμματισμός SoC FPGA

Στα πλαίσια του δεύτερου μέρους της εργαστηριακής άσκησης μας ζητείται να συνδέσουμε το φίλτρο Debayering που έχουμε ήδη υλοποιήσει στο Μέρος Α΄ της άσκησης με AXI4-Stream διεπαφές (AXI4-Stream Slave/Master Interface). Η διασύνδεση των σημάτων εισόδου και εξόδου του φίλτρου με την AXI διεπαφή πρέπει να υλοποιηθεί όπως φαίνεται στο Σχήμα 6:

- `Slave_TDATA[7:0]   = pixel[7:0]`
- `Slave_TVALID       = valid_in`
- `Master_TDATA[32:0] = "00000000" & R[7:0] & G[7:0] & B[7:0]`
- `Master_TVALID      = valid_out`
- `Master_TLAST       = image_finished`



Σχήμα 6: Αρχιτεκτονική Debayering φίλτρου με διεπαφή AXI4-Stream.

Επιπλέον πρέπει να υλοποιήσουμε την ανάλογη εφαρμογή λογισμικού για την αποστολή των σημάτων εισόδου και την λήψη των σημάτων εξόδου του φίλτρου από τον ενσωματωμένο επεξεργαστή. Η επικοινωνία μεταξύ PS-PL θα υλοποιείται μέσω Direct-Memory Access (DMA) λογικής. Η εφαρμογή λογισμικού θα είναι υπεύθυνη για την προετοιμασία του DMA ώστε να στέλνει τα pixel προς το φίλτρο και να δέχεται τα αποτελέσματα. Επίσης, η εφαρμογή λογισμικού θα πρέπει να υλοποιεί και reference software το οποίο θα υπολογίζει και αυτό τα αποτελέσματα του Debayering φίλτρο και θα τα συγκρίνει με τα αποτελέσματα που θα λαμβάνονται από το FPGA.

Ακολουθεί ο κώδικας για το hardware και για το software μαζί με τον σχολιασμό.

Hardware

Ο κώδικας που υλοποιεί το top module του hardware φαίνεται παρακάτω:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dvlsi2021_lab5_top is
  port (
        DDR_cas_n         : inout STD_LOGIC;
        DDR_cke           : inout STD_LOGIC;
        DDR_ck_n          : inout STD_LOGIC;
        DDR_ck_p          : inout STD_LOGIC;
        DDR_cs_n          : inout STD_LOGIC;
        DDR_reset_n       : inout STD_LOGIC;
        DDR_odt           : inout STD_LOGIC;
        DDR_ras_n         : inout STD_LOGIC;
        DDR_we_n          : inout STD_LOGIC;
        DDR_ba            : inout STD_LOGIC_VECTOR( 2 downto 0);
        DDR_addr          : inout STD_LOGIC_VECTOR(14 downto 0);
        DDR_dm            : inout STD_LOGIC_VECTOR( 3 downto 0);
        DDR_dq            : inout STD_LOGIC_VECTOR(31 downto 0);
        DDR_dqs_n         : inout STD_LOGIC_VECTOR( 3 downto 0);
        DDR_dqs_p         : inout STD_LOGIC_VECTOR( 3 downto 0);
        FIXED_IO_mio      : inout STD_LOGIC_VECTOR(53 downto 0);
        FIXED_IO_ddr_vrn  : inout STD_LOGIC;
        FIXED_IO_ddr_vrp  : inout STD_LOGIC;
        FIXED_IO_ps_srstb : inout STD_LOGIC;
        FIXED_IO_ps_clk   : inout STD_LOGIC;
        FIXED_IO_ps_porb  : inout STD_LOGIC
       );
end entity; -- dvlsi2021_lab5_top

architecture arch of dvlsi2021_lab5_top is

  component design_1_wrapper is
    port (
          DDR_cas_n         : inout STD_LOGIC;
          DDR_cke           : inout STD_LOGIC;
          DDR_ck_n          : inout STD_LOGIC;
          DDR_ck_p          : inout STD_LOGIC;
          DDR_cs_n          : inout STD_LOGIC;
          DDR_reset_n       : inout STD_LOGIC;
          DDR_odt           : inout STD_LOGIC;
```

```
          DDR_ras_n         : inout STD_LOGIC;
          DDR_we_n          : inout STD_LOGIC;
          DDR_ba            : inout STD_LOGIC_VECTOR( 2 downto 0);
          DDR_addr          : inout STD_LOGIC_VECTOR(14 downto 0);
          DDR_dm            : inout STD_LOGIC_VECTOR( 3 downto 0);
          DDR_dq            : inout STD_LOGIC_VECTOR(31 downto 0);
          DDR_dqs_n         : inout STD_LOGIC_VECTOR( 3 downto 0);
          DDR_dqs_p         : inout STD_LOGIC_VECTOR( 3 downto 0);
          FIXED_IO_mio      : inout STD_LOGIC_VECTOR(53 downto 0);
          FIXED_IO_ddr_vrn  : inout STD_LOGIC;
          FIXED_IO_ddr_vrp  : inout STD_LOGIC;
          FIXED_IO_ps_srstb : inout STD_LOGIC;
          FIXED_IO_ps_clk   : inout STD_LOGIC;
          FIXED_IO_ps_porb  : inout STD_LOGIC;
          ------------------------------------------------------------------------
---
          ------------------------------------------------ PL (FPGA) COMMON
INTERFACE
          ACLK                                : out STD_LOGIC;
          ARESETN                             : out STD_LOGIC_VECTOR(0 to 0);
          ------------------------------------------------------------------------
-------------
          -- PS2PL-DMA AXI4-STREAM MASTER INTERFACE TO ACCELERATOR AXI4-STREAM
SLAVE INTERFACE
          M_AXIS_TO_ACCELERATOR_tdata         : out STD_LOGIC_VECTOR(7 downto 0);
          M_AXIS_TO_ACCELERATOR_tkeep         : out STD_LOGIC_VECTOR( 0    to 0);
          M_AXIS_TO_ACCELERATOR_tlast         : out STD_LOGIC;
          M_AXIS_TO_ACCELERATOR_tready        : in  STD_LOGIC;
          M_AXIS_TO_ACCELERATOR_tvalid        : out STD_LOGIC;
          ------------------------------------------------------------------------
-------------
          -- ACCELERATOR AXI4-STREAM MASTER INTERFACE TO PL2P2-DMA AXI4-STREAM
SLAVE INTERFACE
          S_AXIS_S2MM_FROM_ACCELERATOR_tdata  : in  STD_LOGIC_VECTOR(31 downto
0);
          S_AXIS_S2MM_FROM_ACCELERATOR_tkeep  : in  STD_LOGIC_VECTOR( 3 downto
0);
          S_AXIS_S2MM_FROM_ACCELERATOR_tlast  : in  STD_LOGIC;
          S_AXIS_S2MM_FROM_ACCELERATOR_tready : out STD_LOGIC;
          S_AXIS_S2MM_FROM_ACCELERATOR_tvalid : in  STD_LOGIC
         );
  end component design_1_wrapper;

  constant N : integer := 1024;
```

```
  component DEBAYERING
      generic( N: integer := 1024 );
      port (
          clk :       IN STD_LOGIC;
          rst_n :     IN STD_LOGIC_vector(0 downto 0);
          new_image : IN STD_LOGIC;
          valid_in :  IN STD_LOGIC;
          pixel :     IN STD_LOGIC_VECTOR(7 DOWNTO 0);
          R : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
          G : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
          B : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
          valid_out:      OUT STD_LOGIC;
          image_finished: OUT STD_LOGIC
      );
  end component;

-----------------------------------------
-- INTERNAL SIGNAL & COMPONENTS DECLARATION

  signal aclk    : std_logic;
  signal aresetn : std_logic_vector(0 to 0);

  signal tmp_tdata  : std_logic_vector(7 downto 0);
  signal tmp_tkeep  : std_logic_vector(0 downto 0);
  signal tmp_tlast  : std_logic;
  signal tmp_tready : std_logic;
  signal tmp_tvalid : std_logic;

  signal valid_in: std_logic;
  signal new_image: std_logic;
  signal pixel: std_logic_vector(7 downto 0);
  signal valid_out: std_logic;
  signal R,G,B : std_logic_vector(7 downto 0);
  signal image_finished: std_logic;
  signal tmp_tdata32: std_logic_vector(31 downto 0);

begin

  tmp_tdata32 <= "00000000"&R&G&B;

  ACCELERATOR: DEBAYERING
    port map (
              new_image=>'1',
              clk=>aclk,
              rst_n=>aresetn,
```

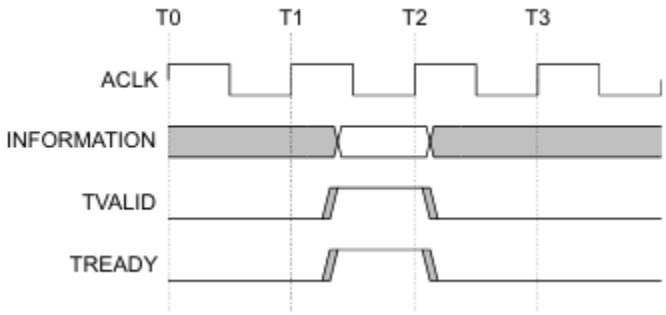
```
              valid_in=>valid_in,
              pixel=>pixel,
              valid_out=>valid_out,
              image_finished=>image_finished,
              R=>R,
              G=>G,
              B=>B
             );
  PROCESSING_SYSTEM_INSTANCE : design_1_wrapper
    port map (
              DDR_cas_n         => DDR_cas_n,
              DDR_cke           => DDR_cke,
              DDR_ck_n          => DDR_ck_n,
              DDR_ck_p          => DDR_ck_p,
              DDR_cs_n          => DDR_cs_n,
              DDR_reset_n       => DDR_reset_n,
              DDR_odt           => DDR_odt,
              DDR_ras_n         => DDR_ras_n,
              DDR_we_n          => DDR_we_n,
              DDR_ba            => DDR_ba,
              DDR_addr          => DDR_addr,
              DDR_dm            => DDR_dm,
              DDR_dq            => DDR_dq,
              DDR_dqs_n         => DDR_dqs_n,
              DDR_dqs_p         => DDR_dqs_p,
              FIXED_IO_mio      => FIXED_IO_mio,
              FIXED_IO_ddr_vrn  => FIXED_IO_ddr_vrn,
              FIXED_IO_ddr_vrp  => FIXED_IO_ddr_vrp,
              FIXED_IO_ps_srstb => FIXED_IO_ps_srstb,
              FIXED_IO_ps_clk   => FIXED_IO_ps_clk,
              FIXED_IO_ps_porb  => FIXED_IO_ps_porb,
              ------------------------------------------------------------------------
-------
              ---------------------------------------------- PL (FPGA) COMMON
INTERFACE
              ACLK                              => aclk,    -- clock to
accelerator
              ARESETN                           => aresetn, -- reset to
accelerator, active low
              ------------------------------------------------------------------------
-----------------
              -- PS2PL-DMA AXI4-STREAM MASTER INTERFACE TO ACCELERATOR AXI4-
STREAM SLAVE INTERFACE
              M_AXIS_TO_ACCELERATOR_tdata       => pixel,
              M_AXIS_TO_ACCELERATOR_tkeep       => tmp_tkeep,
```

```
            M_AXIS_TO_ACCELERATOR_tlast         => tmp_tlast,
            M_AXIS_TO_ACCELERATOR_tready        => '1',
            M_AXIS_TO_ACCELERATOR_tvalid        => valid_in,
            ------------------------------------------------------------------
----------------
            -- ACCELERATOR AXI4-STREAM MASTER INTERFACE TO PL2P2-DMA AXI4-
STREAM SLAVE INTERFACE
            S_AXIS_S2MM_FROM_ACCELERATOR_tdata  => tmp_tdata32,
            S_AXIS_S2MM_FROM_ACCELERATOR_tkeep  => "1111",
            S_AXIS_S2MM_FROM_ACCELERATOR_tlast  => image_finished,
            S_AXIS_S2MM_FROM_ACCELERATOR_tready => tmp_tready,
            S_AXIS_S2MM_FROM_ACCELERATOR_tvalid => valid_out
           );

----------------------------
-- COMPONENTS INSTANTIATIONS

end architecture; -- arch
```

Σε κάθε περίπτωση για να μπορέσουν να επικοινωνήσουν μεταξύ τους τα δύο DMA engines μέσω της AXI4-Stream διεπαφής του Debayering θα πρέπει να υπάρξει handshake μεταξύ TVALID και TREADY, όπως φαίνεται παρακάτω:



Από την μεριά του PS2PL στο σήμα TDATA[7:0] περνάμε ως είσοδο τα pixel της εικόνας. Έπειτα ελέγχουμε αν το Slave είναι έτοιμο να δεχτεί δεδομένα με το TREADY και αν αυτά είναι έγκυρα με το TVALID περνώντας τα στο valid_in.

Από την μεριά του PL2PS στο σήμα TDATA[32:0] που διαμορφώνουμε ως εξής: "00000000" & R[7:0] & G[7:0] & B[7:0] βγάζουμε τις τιμές RGB του κάθε pixel. Έπειτα με το σήμα TVALID που λαμβάνεται από το valid_out ελέγχουμε την εγκυρότητα των εξόδων και με το σήμα TLAST του Master λαμβάνουμε το σήμα ότι τελείωσε η επεξεργασία της εικόνας (image_finished).

Software

Ο κώδικας που υλοποιεί το ζητούμενο πρόγραμμα σε C φαίνεται παρακάτω:

```
#include <stdio.h>
#include <unistd.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xparameters_ps.h"
#include "xaxidma.h"
#include "xtime_l.h"
#include "image.h" //test input data

#define TX_DMA_ID XPAR_PS2PL_DMA_DEVICE_ID                      // PS2PL
#define TX_DMA_MM2S_LENGTH_ADDR (XPAR_PS2PL_DMA_BASEADDR + 0x28) // Bytes of data
from PS->PL

#define RX_DMA_ID XPAR_PL2PS_DMA_DEVICE_ID                      // PL2PS
#define RX_DMA_S2MM_LENGTH_ADDR (XPAR_PL2PS_DMA_BASEADDR + 0x58) // Bytes of data
from PL->PS

#define TX_BUFFER (XPAR_DDR_MEM_BASEADDR + 0x10000000) // 0 + 256MByte
#define RX_BUFFER (XPAR_DDR_MEM_BASEADDR + 0x18000000) // 0 + 384MByte

/* User application global variables & defines */
#define N 0x400           // N=1024
#define IMG_SIZE 0x100000 // N*N=1048576

/* Device instance definitions*/
XAxiDma TXAxiDma; // PS2PL DMA
XAxiDma RXAxiDma; // PL2PS DMA

int main()
{
    Xil_DCacheDisable();
    XTime preExecCyclesFPGA = 0;
    XTime postExecCyclesFPGA = 0;
    XTime preExecCyclesSW = 0;
    XTime postExecCyclesSW = 0;
    // User application local variables
    init_platform();
    /************************DEVICE
INITIALIZATION*****************************/
    u8 *TxBufferPtr;
```

```
    u32 *RxBufferPtr;
    TxBufferPtr = (u8 *)TX_BUFFER;  // 8bit input
    RxBufferPtr = (u32 *)RX_BUFFER; // 32bit output
    // Step 1: Initialize TX-DMA Device (PS->PL)
    /* Configure & Initialize PS2PL */
    XAxiDma_Config *TXCfgPtr;

    TXCfgPtr = XAxiDma_LookupConfig(TX_DMA_ID);

    XAxiDma_CfgInitialize(&TXAxiDma, TXCfgPtr);

    XAxiDma_IntrDisable(&TXAxiDma, XAXIDMA_IRQ_ALL_MASK, // disable interrupts
                        XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_IntrDisable(&TXAxiDma, XAXIDMA_IRQ_ALL_MASK, // we want polling mode
                        XAXIDMA_DMA_TO_DEVICE);

    // Step 2: Initialize RX-DMA Device (PL->PS)
    /* Configure & Initialize PS2PL */
    XAxiDma_Config *RXCfgPtr;

    RXCfgPtr = XAxiDma_LookupConfig(RX_DMA_ID);

    XAxiDma_CfgInitialize(&RXAxiDma, RXCfgPtr);

    XAxiDma_IntrDisable(&RXAxiDma, XAXIDMA_IRQ_ALL_MASK, // disable interrupts
                        XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_IntrDisable(&RXAxiDma, XAXIDMA_IRQ_ALL_MASK, // we want polling mode
                        XAXIDMA_DMA_TO_DEVICE);

    for (int Index = 0; Index < IMG_SIZE; Index++)
    { // Data to accelerator
        TxBufferPtr[Index] = pixels[Index];
    }
    for (int Index = 0; Index < 10; Index++)
    {
        RxBufferPtr[Index] = Index;
    }
    /*****************************************************************************
*/
    XTime_GetTime(&preExecCyclesFPGA);
    // Step 3 : Perform FPGA processing
    //      3a: Setup RX-DMA transaction
    //      3b: Setup TX-DMA transaction
    //      3c: Wait for TX-DMA & RX-DMA to finish
    XAxiDma_SimpleTransfer(&RXAxiDma, (UINTPTR)RxBufferPtr,
```

```
                          IMG_SIZE * 4, XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_SimpleTransfer(&TXAxiDma, (UINTPTR)TxBufferPtr,
                          IMG_SIZE, XAXIDMA_DMA_TO_DEVICE);

    if (XAxiDma_Busy(&RXAxiDma, XAXIDMA_DEVICE_TO_DMA))
        print("RX DEVICE TO DMA BUSY\r\n");
    if (XAxiDma_Busy(&RXAxiDma, XAXIDMA_DMA_TO_DEVICE))
        print("RX DMA TO DEVICE BUSY\r\n");
    if (XAxiDma_Busy(&TXAxiDma, XAXIDMA_DEVICE_TO_DMA))
        print("TX DEVICE TO DMA BUSY\r\n");
    if (XAxiDma_Busy(&TXAxiDma, XAXIDMA_DMA_TO_DEVICE))
        print("TX DMA TO DEVICE BUSY\r\n");

    XTime_GetTime(&postExecCyclesFPGA);
    /*************************************************************************
*/
    XTime_GetTime(&preExecCyclesSW);
    // Step 5: Perform SW processing
    int *r_sw = (int *)malloc(N * N * sizeof(int));
    int *g_sw = (int *)malloc(N * N * sizeof(int));
    int *b_sw = (int *)malloc(N * N * sizeof(int));
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            int right = (j < N - 1) ? (j + 1) : 0;
            int left = (j > 0) ? (j - 1) : 0;
            int up = (i > 0) ? (i - 1) : 0;
            int down = (i < N - 1) ? (i + 1) : 0;

            int r = (j < N - 1) ? pixels[right + N * i] : 0;
            int l = (j > 0) ? pixels[i * N + left] : 0;
            int u = (i > 0) ? pixels[up * N + j] : 0;
            int d = (i < N - 1) ? pixels[down * N + j] : 0;

            int d1 = ((i > 0) && (j > 0)) ? pixels[up * N + left] : 0;
            int d2 = ((i > 0) && j < (N - 1)) ? pixels[up * N + right] : 0;
            int d3 = ((i < N - 1) && (j > 0)) ? pixels[down * N + left] : 0;
            int d4 = ((i < N - 1) && (j < N - 1)) ? pixels[down * N + right] : 0;
            int cc = pixels[i * N + j];
            if (i % 2)
            {
                if (j % 2)
                { // green
                    r_sw[i * N + j] = (r + l) / 2;
```

```
                    g_sw[i * N + j] = cc;
                    b_sw[i * N + j] = (u + d) / 2;
                }
                else
                { // red
                    r_sw[i * N + j] = cc;
                    g_sw[i * N + j] = (u + d + r + l) / 4;
                    b_sw[i * N + j] = (d1 + d2 + d3 + d4) / 4;
                }
            }
            else
            {
                if (j % 2)
                { // blue
                    r_sw[i * N + j] = (d1 + d2 + d3 + d4) / 4;
                    g_sw[i * N + j] = (u + d + r + l) / 4;
                    b_sw[i * N + j] = cc;
                }
                else
                { // green
                    r_sw[i * N + j] = (u + d) / 2;
                    g_sw[i * N + j] = cc;
                    b_sw[i * N + j] = (r + l) / 2;
                }
            }
        }
    }
    XTime_GetTime(&postExecCyclesSW);
    /*****************************************************************************
*/
    // Step 6: Compare FPGA and SW results
    //     6a: Report total percentage error
    //     6b: Report FPGA execution time in cycles (use preExecCyclesFPGA and
postExecCyclesFPGA)
    //     6c: Report SW execution time in cycles (use preExecCyclesSW and
postExecCyclesSW)
    //     6d: Report speedup (SW_execution_time / FPGA_exection_time)
    int *r_hw = (int *)malloc(N * N * sizeof(int));
    int *g_hw = (int *)malloc(N * N * sizeof(int));
    int *b_hw = (int *)malloc(N * N * sizeof(int));
    int err = 0;
    int err_per, FPGA_cycles, SW_cycles, speedup;
    for (int i = 0; i < IMG_SIZE; i++)
    {
        r_hw[i] = (RxBufferPtr[i] >> 16) & 0xff;
```

```
        g_hw[i] = (RxBufferPtr[i] >> 8) & 0xff;
        b_hw[i] = RxBufferPtr[i] & 0xff;
        if (r_hw[i] == r_sw[i] && g_hw[i] != g_sw[i] && b_hw[i] == b_sw[i])
            err++;
    }
    err_per = (err / IMG_SIZE) * 100;
    FPGA_cycles = postExecCyclesFPGA - preExecCyclesFPGA;
    SW_cycles = postExecCyclesSW - preExecCyclesSW;
    speedup = SW_cycles / FPGA_cycles;
    printf("Total Percentage error: %d %% \n", err_per);
    printf("FPGA execution time in cycles: %d \n", FPGA_cycles);
    printf("SW execution time in cycles: %d \n", SW_cycles);
    printf("Speedup: %d \n", speedup);
    cleanup_platform();
    return 0;
}
```